Hash-Based Digital Signatures: From Lamport OTS to Merkle Signature Schemes

Applied Cryptography (CS 232) — Project Report

Aicha Slaitane
aicha.slaitane@kaust.edu.sa

Abdullah Algethami
abdullah.algethami@kaust.edu.sa

Abstract

We design, implement, and evaluate a complete hash-based digital signature system in Python. Starting from the foundational Lamport One-Time Signature (OTS) scheme, we progress through Winternitz OTS (WOTS) and culminate in a full Merkle Signature Scheme (MSS) that lifts the one-time restriction. We conduct systematic benchmarks across a wide parameter space—varying the Winternitz parameter $w \in \{4, \ldots, 256\}$ and tree heights $h \in \{1, \ldots, 8\}$—to quantify the signature-size versus computation-time trade-off. We demonstrate the critical key-reuse vulnerability of Lamport signatures through a quantitative attack simulation, recovering the full 512-block secret key after only 20 reuses and validating the theoretical forgery probability $(1 - 2^{-k})^{256}$ against empirical data across 200 independent trials per configuration. Finally, we extend the construction to a two-layer *hypertree*, the same multi-layer Merkle aggregation idea used internally by SPHINCS+/SLH-DSA, and show that it reduces initial KeyGen time by an order of magnitude at matched signing capacity (e.g., 8× faster at $N=256$ and 16× faster at $N=1024$).

1 Introduction

Digital signatures are a cornerstone of modern information security, providing authentication, integrity, and non-repudiation. Classical schemes such as RSA and ECDSA derive their security from the computational hardness of integer factorization and the discrete logarithm problem, respectively. However, Shor's algorithm [1] demonstrates that a sufficiently powerful quantum computer can solve both problems in polynomial time, rendering these schemes insecure in a post-quantum world.

Hash-based signatures offer a fundamentally different security foundation. Their security rests on a single, well-studied assumption: the *one-wayness* and *(second-)preimage resistance* of the underlying cryptographic hash function. The best known quantum speedup against generic hash functions is Grover's algorithm, which provides only a quadratic advantage—reducing an n-bit hash from 2^n to $2^{n/2}$ security. For SHA-256, this yields approximately 128 bits of quantum security, aligning with NIST's Category-1 post-quantum requirement. This robustness is precisely why NIST standardized the hash-based scheme SLH-DSA (SPHINCS+) alongside the lattice-based ML-DSA in 2024.

In this project, we implement the fundamental building blocks that underpin modern hash-based signature standards: Lamport OTS, Winternitz OTS, and the Merkle Signature Scheme. All implementations use SHA-256 and are written in pure Python with no external cryptographic libraries. We accompany each construction with unit tests, systematic benchmarks, and a thorough security analysis of the key-reuse vulnerability inherent in OTS schemes.

2 Methods

2.1 Lamport One-Time Signatures

Proposed by Lamport in 1979 [2], the scheme generates a secret key consisting of n pairs of random blocks $\{(\mathrm{SK}[i][0], \mathrm{SK}[i][1])\}_{i=0}^{n-1}$, where n is the hash output length in bits (256 for SHA-256) and each block is 32 bytes. The public key is the element-wise hash: $\mathrm{PK}[i][b] = H(\mathrm{SK}[i][b])$. To sign a message m, the signer computes $h = H(m)$ and reveals $\mathrm{SK}[i][h_i]$ for each bit h_i. Verification rehashes each revealed block and compares against the public key. The resulting signature contains 256 blocks of 32 bytes each, totaling 8192 bytes, and the public key is twice that size (16384 bytes) since it stores both hash images per row.

2.2 Winternitz OTS (WOTS)

WOTS generalizes Lamport by processing the message hash in base-w digits rather than individual bits, dramatically reducing signature size. Each digit is signed by its own independent hash chain. The secret key consists of $\ell = \ell_1 + \ell_2$ random start values (chains). For a 256-bit hash (e.g., SHA-256) and $w = 16$, the message requires $\ell_1 = 256/4 = 64$ chains. To prevent forgery where an attacker hashes a revealed value forward to claim a higher digit, WOTS appends a checksum $C = \sum_i (w - 1 - d_i)$. With $w = 16$, the maximum sum of 64 digits is $64 \times 15 = 960$, requiring $\ell_2 = 3$ additional base-16 chains to encode ($16^3 = 4096$). Thus, the full key uses exactly $64 + 3 = 67$ parallel chains. The public key is obtained by hashing each secret block $w - 1$ times to reach the chain end: $\mathrm{PK}[i] = H^{w-1}(\mathrm{SK}[i])$. To sign digit d_i, the signer re-

veals the value at position d_i in the chain. The verifier finishes the chain by applying $w - 1 - d_i$ additional hashes and checks against the public key end. This reduces signature size significantly (e.g., 2,144 B vs. 8,192 B for Lamport) but increases computation since each chain requires up to $w - 1$ hash evaluations.

2.3 Merkle Signature Scheme

OTS schemes are inherently single-use. The Merkle tree construction [3] overcomes this by generating $N = 2^h$ independent OTS keypairs, hashing each OTS public key into a leaf node, and building a binary hash tree whose root serves as the long-term public key. A signature for the i-th message includes: (a) the OTS signature, (b) the OTS public key, and (c) an *authentication path*—the h sibling hashes along the path from leaf i to the root. The verifier checks the OTS signature and reconstructs the root from the leaf using the authentication path, accepting if and only if the reconstructed root matches the published public key. A `next_leaf` counter enforces strict one-time usage of each leaf, raising an error upon exhaustion.

2.4 Multi-Layer Merkle Trees (Hypertrees)

Section 3 (and Figure 3a in particular) exposed the dominant practical limitation of a flat MSS: KeyGen scales as $O(2^h)$, which becomes prohibitive once we want signing capacities of 2^{16} or beyond. We address this by implementing a two-layer *hypertree*, the same multi-layer Merkle construction used internally by SPHINCS+/SLH-DSA.

A two-layer hypertree consists of one *top tree* of height h_{top} and multiple *bottom trees* of height h_{bot}. The top tree is built *immediately* during initial key generation. Its OTS keypairs are generated deterministically from a secret seed, and its leaves are simply the hashes of these Top OTS public keys. Crucially, the top tree leaves do not contain or depend on the roots of the bottom trees, allowing the top tree to be fully constructed in advance. The root of this top tree serves as the global public key.

The decisive property is that bottom trees are built *lazily*. The cryptographic link between the top tree and a bottom tree is forged dynamically when a signature is requested. The process works as follows:

1. **Materialization**: Based on the message index, the specific bottom tree needed for that index is generated, and its root hash is computed.
2. **The Link Signature**: This bottom-tree root is treated as a "message" and is signed using the Top OTS secret key corresponding to the leaf directly above that bottom tree. Because the Top OTS public keys are already committed to in the global top root, this signature securely binds the lazily generated bottom tree into the trusted global structure.
3. **The Final Bundle**: The resulting hypertree signature bundles four components: (1) the bottom-tree OTS signature on the actual message, (2) the bottom-tree authentication path to reconstruct the bottom root, (3) the Top OTS link signature on the reconstructed bottom root, and (4) the top-tree authentication path to climb from the top leaf to the global top root.

The total signing capacity is $N = 2^{h_{\text{top}}+h_{\text{bot}}}$. This lazy linking turns the up-front cost from $O(2^h)$ into $O(2^{h_{\text{top}}})$—a square-root reduction when $h_{\text{top}} \approx h_{\text{bot}}$. Verification simply walks this chain: it verifies the bottom-tree signature to get the bottom root, then verifies the top-tree signature on that bottom root to climb to the global top root.

3 Experimental Results

All benchmarks were collected on a local Linux workstation using SHA-256. Timing measurements are reported as mean ± variance over 5 independent runs to capture both the typical execution time and its stability. A single fixed string (`"Post-quantum cryptography is exciting"`) was signed repeatedly to ensure a fair, controlled comparison across all configurations.

3.1 Lamport Key-Reuse Attack

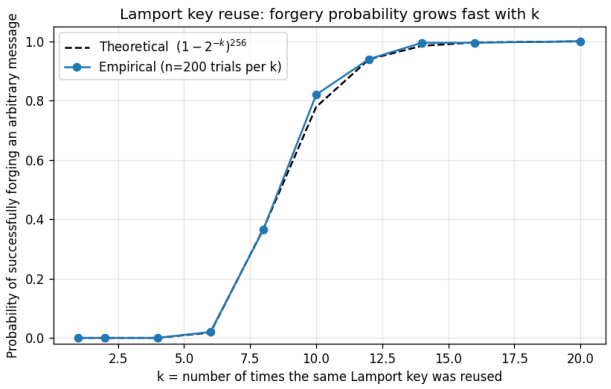
3.1.1 Threat Model and Theoretical Analysis

A Lamport secret key comprises $256 \times 2 = 512$ blocks. Each signature reveals exactly 256 blocks (one per row, selected by the corresponding message-hash bit). After k reuses with independent random messages, the probability that a *specific* block remains unrevealed is $(1/2)^k$. To forge a signature on an arbitrary target, the attacker needs all 256 blocks selected by the target's hash. The forgery probability is:

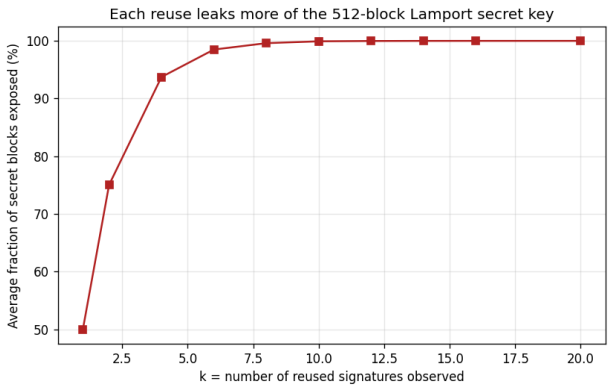
$$P_{\text{forge}}(k) = \left(1 - 2^{-k}\right)^{256} \tag{1}$$

We verified Eq. (1) empirically by running 200 independent trials per value of k, each with a fresh keypair (Table 1). As shown in Figure 1a, the empirical forgery rate tracks the theoretical curve closely. After only ∼10 reuses, the attacker succeeds with >78% probability; after 15, success is essentially certain. The attack is entirely *passive*—the attacker only needs to observe legitimate signatures without any interaction with the signer.

Figure 1b provides a complementary view: it plots the average fraction of the 512-block secret key exposed as a function of k. By $k = 8$, over 99.6% of blocks are revealed, and by $k = 12$ the entire key is typically exposed. This highlights that even a small number of reuses leaks nearly all secret material.



(a) Forgery success rate vs. k.



(b) Fraction of secret-key blocks exposed vs. reuse count k.

Figure 1: Empirical and theoretical metrics for the Lamport key-reuse attack over $k \in \{1, \ldots, 20\}$ reuses (200 independent trials per configuration).

k	Theoretical	Empirical	Exposed
1	0.000	0.000	50.0%
4	0.000	0.000	93.9%
6	0.018	0.030	98.5%
8	0.367	0.385	99.6%
10	0.779	0.835	99.9%
12	0.939	0.950	100.0%
14	0.984	1.000	100.0%
20	1.000	1.000	100.0%

Table 1: Forgery probability vs. key reuses k. "Exposed" is the average fraction of all 512 secret blocks revealed to the attacker.

3.1.2 Full Secret Key Recovery

We explicitly demonstrated secret-key recovery by signing 20 random messages with the same Lamport key. All 512 blocks were recovered byte-exact, as verified against the original key. The full recovered key was exported to JSON for inspection. Once in possession of the complete secret key, the attacker can forge a valid signature on *any* message.

3.2 OTS Primitive Comparison

We benchmarked Lamport OTS and WOTS across key Winternitz parameters ($w = 4, 16, 64, 256$). Table 2 summarizes the key generation, signing, verification times, and key/signature sizes.

The Winternitz trade-off is clearly visible in both Table 2 and Figures 2a–2b. As w increases, signature size decreases logarithmically (Figure 2a): WOTS with $w=256$ is 7.5× smaller than Lamport (1088 B vs. 8192 B). This reduction arises because each doubling of w groups more bits per digit, reducing the number of hash chains ℓ_1 by a factor proportional to $\log_2 w$. However, computation cost grows linearly with w (Figure 2b), since each chain is $w-1$ hashes long: WOTS with $w=256$ is approximately 75× slower to sign than Lamport. Note that Lamport's signing is extremely fast (0.03 ms) because it involves no hash computation at all—only selecting and copying precomputed blocks.

3.3 MSS Height Sweep

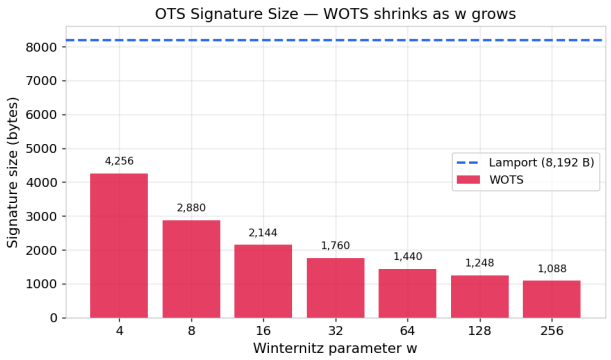
Table 3 reports MSS performance for tree heights $h =$ 1 to 8 using both Lamport and WOTS ($w=16$) as the leaf OTS.

Three key observations emerge from Table 3 and Figures 3a–3d. First, as shown in Figure 3a, KeyGen scales as $O(2^h)$: going from $h = 4$ to $h = 8$ increases the cost by roughly 16×, confirming that the dominant expense is materializing a full OTS keypair at every leaf. Second, Sign times remain effectively constant with respect to h (Figure 3b), because the additional h hash operations for the authentication path are negligible compared to the underlying OTS work. Verify times exhibit the exact same constant scaling (Figure 3c). Third, signature size grows by only 32 bytes per tree level (Figure 3d)—one SHA-256 sibling hash—meaning that doubling the signing capacity ($h \to h+1$) costs a trivial 32 B overhead per signature. Across all heights, WOTS leaves produce roughly 5.5× smaller MSS signatures than Lamport leaves (∼4.5 KB vs. ∼24.7 KB).

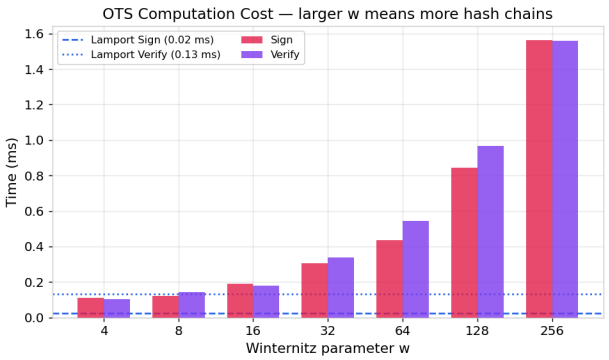
3.4 Parameter Tuning: MSS with Varying w

Fixing $h=4$ (16 leaves), we varied the WOTS parameter to study the size-vs-speed trade-off within the full MSS context (Table 4). Going from $w=4$ to $w=256$ shrinks signatures by 3.7× (8644 B to 2308 B) but increases total computation (sign + verify) by 13×. Figure 4 visualizes this Pareto frontier, showing the smooth and predictable nature of the trade-off curve. The "knee" of the curve sits near $w=16$, which is why real-world standards (XMSS, SPHINCS+) adopt this value.

Based on the complete sweep, Table 5 summarizes

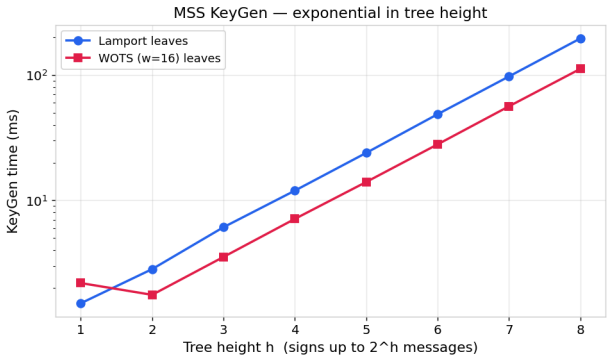


(a) Signature size vs. w.

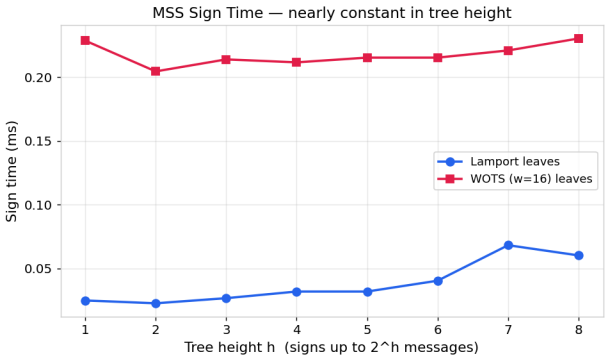


(b) Sign and verify times vs. w.

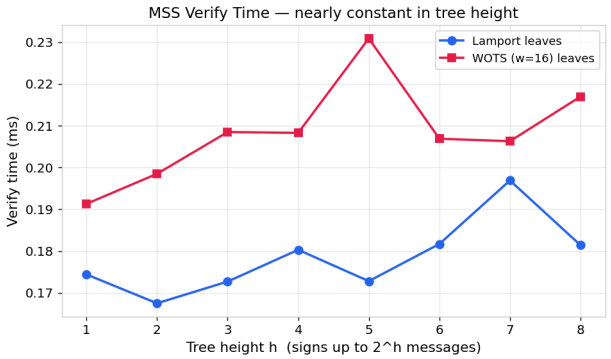
Figure 2: OTS performance and scaling trade-offs across Winternitz parameters $w \in \{4, 16, 64, 256\}$ compared to Lamport OTS. Line in (b) shows Lamport verification time (0.13 ms) and signing time (0.02 ms) for reference.



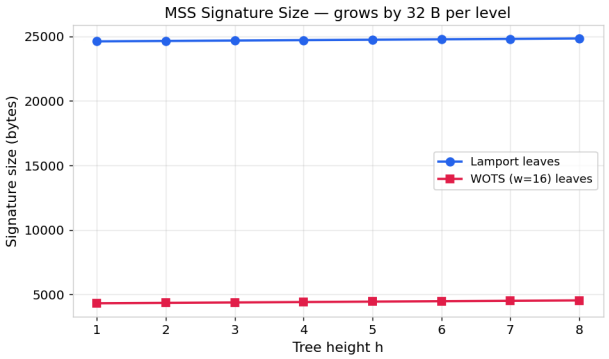
(a) MSS KeyGen time vs. tree height on a log scale.



(b) MSS Sign time vs. height.



(c) MSS Verify time vs. height.



(d) MSS signature size vs. height.

Figure 3: MSS performance metrics across tree heights $h \in \{1, \ldots, 8\}$ for both Lamport and WOTS ($w = 16$) leaf backends.

Scheme	KG (ms)	Sign (ms)	Verify (ms)	Sig (B)	PK (B)
Lamport	0.74 ± 0.013	0.02 ± 0.000	0.13 ± 0.002	8192	16384
WOTS $w=4$	0.29 ± 0.000	0.11 ± 0.000	0.10 ± 0.000	4256	4256
WOTS $w=16$	0.40 ± 0.000	0.19 ± 0.000	0.18 ± 0.000	2144	2144
WOTS $w=64$	1.06 ± 0.011	0.43 ± 0.000	0.55 ± 0.000	1440	1440
WOTS $w=256$	3.00 ± 0.006	1.56 ± 0.021	1.56 ± 0.014	1088	1088

Table 2: OTS performance across Winternitz parameters. KG = KeyGen, Vrfy = Verify, Sig = signature size, PK = public key size.

Config	N	KG (ms)	Sign (ms)	Sig (B)
$h=4$, Lamp.	16	11.96 ± 0.01	0.03 ± 0.00	24640
$h=6$, Lamp.	64	48.73 ± 0.71	0.04 ± 0.00	24704
$h=8$, Lamp.	256	196.28 ± 10.19	0.06 ± 0.00	24768
$h=4$, WOTS	16	7.11 ± 0.03	0.21 ± 0.00	4420
$h=6$, WOTS	64	27.99 ± 0.17	0.22 ± 0.00	4484
$h=8$, WOTS	256	112.17 ± 1.13	0.23 ± 0.00	4548

Table 3: MSS performance vs. tree height ($N = 2^h$ leaves).

w	KG (ms)	Sign (ms)	Vrfy (ms)	Sig (B)
4	4.95	0.16	0.18	8644
16	9.09	0.30	0.30	4420
64	23.52	0.65	0.84	3012
256	70.59	2.26	2.26	2308

Table 4: MSS ($h=4$) performance vs. Winternitz parameter w.

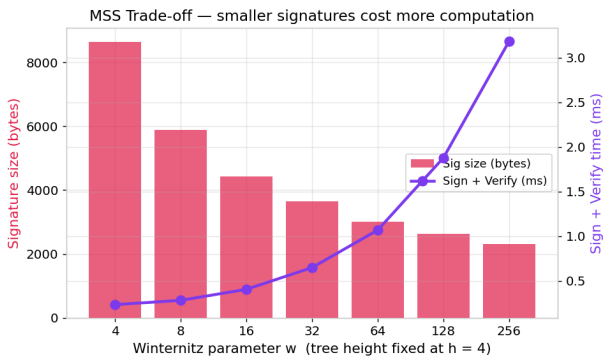


Figure 4: MSS size-vs-speed trade-off at $h=4$. The knee near $w = 16$ marks the practical sweet spot adopted by XMSS and SPHINCS+.

optimal parameter choices for four representative deployment priorities.

Priority	Config	Sig	S+V	N
Min size	$w=256, h=4$	2308 B	4.52 ms	16
Min compute	Lamp., $h=4$	24708 B	0.33 ms	16
Balanced	$w=16, h=5$	4452 B	0.61 ms	32
Max msgs	$w=16, h=8$	4548 B	0.61 ms	256

Table 5: Recommended configurations for different priorities (S+V = Sign + Verify time, N = max messages).

3.5 Hypertree Performance

Table 6 compares the two-layer hypertree against a flat MSS at matched signing capacities. WOTS ($w=16$) leaves are used throughout. Figure 5 visualizes the KeyGen comparison.

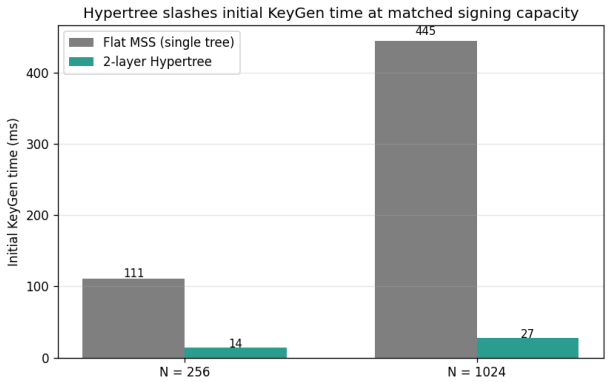


Figure 5: Initial KeyGen time: flat MSS vs 2-layer hypertree at matched signing capacities. The hypertree is 8.2× faster at $N=256$ and 15.6× faster at $N=1024$, confirming the predicted $O(\sqrt{N})$ behavior.

The data confirms three predictions. First, KeyGen time drops by roughly the expected $\sqrt{N}$ factor (8.2× at $N = 256$, 15.6× at $N = 1024$). Second, single-message Sign time is essentially unchanged, because signing still costs one bottom-tree MSS sign plus a precomputed top-tree certificate. Third, signature size roughly doubles (∼4.5 KB to ∼8.9 KB), since each hypertree signature now carries two MSS signatures instead of one—the standard size-vs-cost trade-off of multi-layer Merkle constructions. This is precisely the

Scheme	N	KG (ms)	Sign (ms)	Vrfy (ms)	Sig (B)
Flat MSS, $h=8$	256	110.7 ± 0.6	0.20	0.23	4548
Hypertree, $h_t=h_b=4$	256	13.8 ± 0.01	0.17	0.60	8876
Flat MSS, $h=10$	1024	444.9 ± 4.7	0.20	0.24	4612
Hypertree, $h_t=h_b=5$	1024	27.4 ± 0.4	0.19	0.43	8940

Table 6: Hypertree vs flat MSS at matched signing capacities. KG = initial KeyGen time.

engineering trick that lets SPHINCS+ scale to $N=2^{64}$ while keeping initial KeyGen practical.

4 Conclusion

We implemented a complete hash-based signature pipeline from Lamport OTS through WOTS to the Merkle Signature Scheme, extended it to a two-layer hypertree, and supported all of it with comprehensive benchmarks and a quantitative key-reuse attack. Our key findings are:

- WOTS with $w = 16$ offers the best practical balance: 3.8× smaller signatures than Lamport with sign+verify under 0.6 ms.
- Flat MSS KeyGen scales as $O(2^h)$, while signing, verification, and signature size are nearly independent of tree height.
- Lamport key reuse is catastrophic: after ∼10 reuses, forgery succeeds with >78% probability; after 15 reuses, it is essentially certain. We recovered the full 512-block secret key byte-for-byte after only 20 reuses.
- The two-layer hypertree (Section 2.4) reduces initial KeyGen time by 8–16× at matched signing capacity, at the cost of roughly doubling the signature size—the same engineering trick used by SPHINCS+/SLH-DSA.

These results confirm that hash-based signatures are a viable and well-understood path to post-quantum security, requiring only the mild assumption of hash function one-wayness.

References

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," *Proc. 35th Annual Symp. on Foundations of Computer Science (FOCS)*, pp. 124–134, IEEE, 1994.

[2] L. Lamport, "Constructing digital signatures from a one-way function," *SRI International, Technical Report CSL-98*, 1979.

[3] R. C. Merkle, "A certified digital signature," *Advances in Cryptology — CRYPTO '89*, LNCS 435, pp. 218–238, Springer, 1990.

[4] D. J. Bernstein et al., "SPHINCS+: Submission to the NIST post-quantum cryptography standardization project," 2017.